

1. Länge von Läufen in Feldern

3 Punkte

Gegeben ist ein Feld $a[1:n]$ mit $1 \leq n \leq \max$ gemäß folgender Deklarationen:

```
const
  max = ...
type
  IntArray = array[1:max] of int
```

Gesucht ist ein Algorithmus

$\text{Count}(\downarrow a: \text{IntArray} \downarrow n: \text{int} \uparrow l: \text{int})$

der die Länge l des längsten Laufs in a berechnet. Ein *Lauf* ist eine Teilfolge von aufsteigend sortierten Elementen in a .

Beispiel: Für $a = (2, \underline{1}, \underline{3}, \underline{4}, 2, 7, 5, 4, 6)$ muss der Algorithmus $l = 3$ liefern.

2. Mischen von Listen

5 Punkte

Gegeben sind zwei einfach-verkettete Listen a und b gemäß folgender Deklarationen:

```
type ListNodePtr = →ListNode
  ListNode = compound
    data: int
    next: ListNodePtr
  end -- compound
  ListPtr = ListNodePtr
```

Die Knoten der beiden Listen a und b sind entsprechend dem Wert in der Datenkomponente $data$ aufsteigend sortiert.

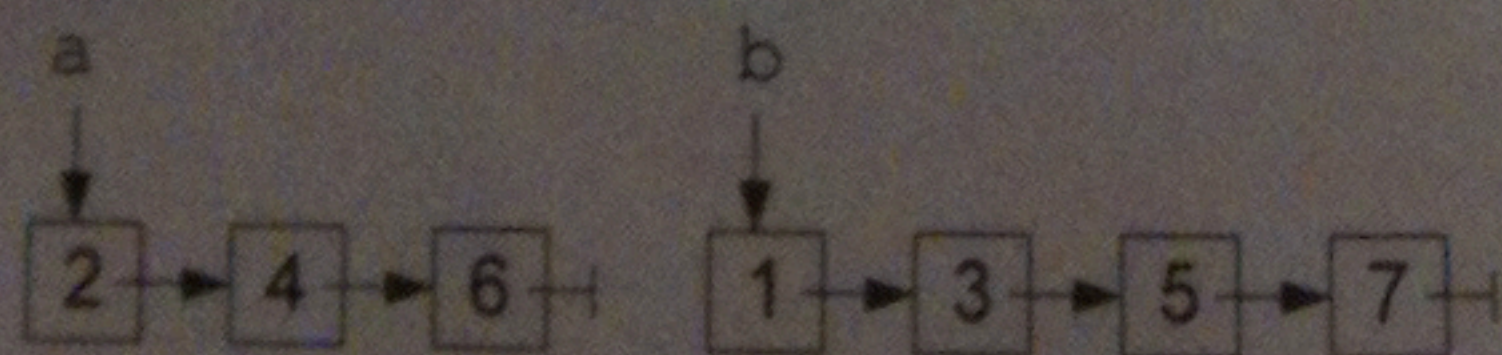
Gesucht ist ein Algorithmus

$\text{Merge}(\downarrow a: \text{ListPtr} \downarrow b: \text{ListPtr} \uparrow c: \text{ListPtr})$

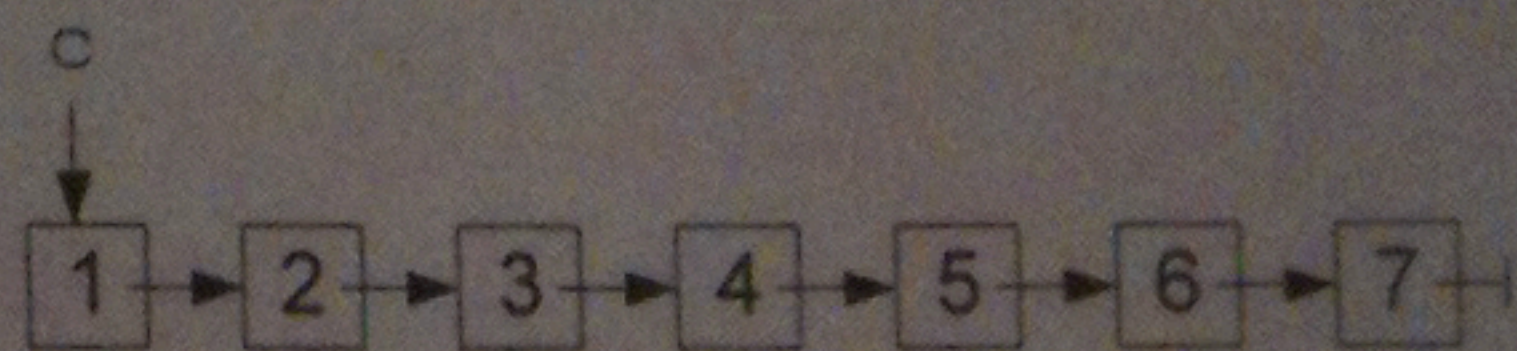
der die Listen a und b zu einer Liste c zusammenmischt, so dass die Knoten in c wieder aufsteigend sortiert sind.

Nebenbedingung: Die Liste c darf nur Knoten der Listen a und b enthalten, es dürfen keine neuen Knoten erzeugt werden.

Beispiel:



Vor dem Aufruf von *Merge*



Nach dem Ablauf von *Merge* (a und b sind leer)

3. Knoten in Ebenen

4 Punkte

Gegeben ist ein Binärbaum auf Basis folgender Deklarationen:

```
type TreeNodePtr = ->TreeNode
  TreeNode = compound
    data: int
    left, right: TreeNodePtr
  end -- compound
  TreePtr = TreeNodePtr
```

Gesucht ist ein Algorithmus

```
NodesAtLevel( $\downarrow$ tree: TreePtr  $\downarrow$ level: int): int
```

der die Anzahl von Knoten auf der Ebene *level* ermittelt.

Hinweis: Gemäß den in der Vorlesung vorgenommenen Definitionen wird jeder Knoten eines Baumes einer bestimmten Ebene im Baum zugeordnet, die Wurzel der Ebene 0, die direkten Nachfolgeknoten der Wurzel der Ebene 1 usw.

4. Teile und Herrsche

3 Punkte

Erläutern Sie mit eigenen Worten, was man unter der Strategie „Teile und Herrsche“ versteht. Nennen Sie zwei Ihnen bekannte Algorithmen, die auf dieser Strategie basieren.

5. Komplexität

2 + 1 Punkte

Gegeben ist der Algorithmus A:

```
A( $\downarrow$ n: int)
  var i, j: int
  begin
    i := n
    while (i > 1) do
      j := 1
      while j < i do
        j := j + 1
      end -- while
      i := i div 2
    end -- while
  end A
```

Bestimmen Sie für diesen Algorithmus

- (a) die Strukturkomplexität *V* und die essentielle Strukturkomplexität *EV* nach McCabe
- (b) die asymptotische Laufzeitkomplexität

6. Sortialgorithmen

3 Punkte

Erläutern Sie mit eigenen Worten die Funktionsweise des *ShellSort*-Verfahrens nach D. L. Shell. Von welchen Faktoren hängt seine Laufzeitkomplexität ab?

7. Rekursion

1 + 1 + 1 Punkte

- (a) Erklären Sie mit eigenen Worten was man unter dem Konzept *Rekursion* versteht.
- (b) Erläutern Sie die Vorteile rekursiver Algorithmen im Vergleich zur iterativen Algorithmen.
- (c) Erläutern Sie die Nachteile rekursiver Algorithmen im Vergleich zur iterativen Algorithmen.